Module 7: Tensor Parallelism

Splitting Matrix Multiplication Inside Each Layer

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. Start from one matrix multiplication
2. Inside a Transformer block
3. Cost and placement

Guiding question

How can several GPUs compute one Transformer layer without reconstructing every intermediate tensor?

What should be clear by the end?

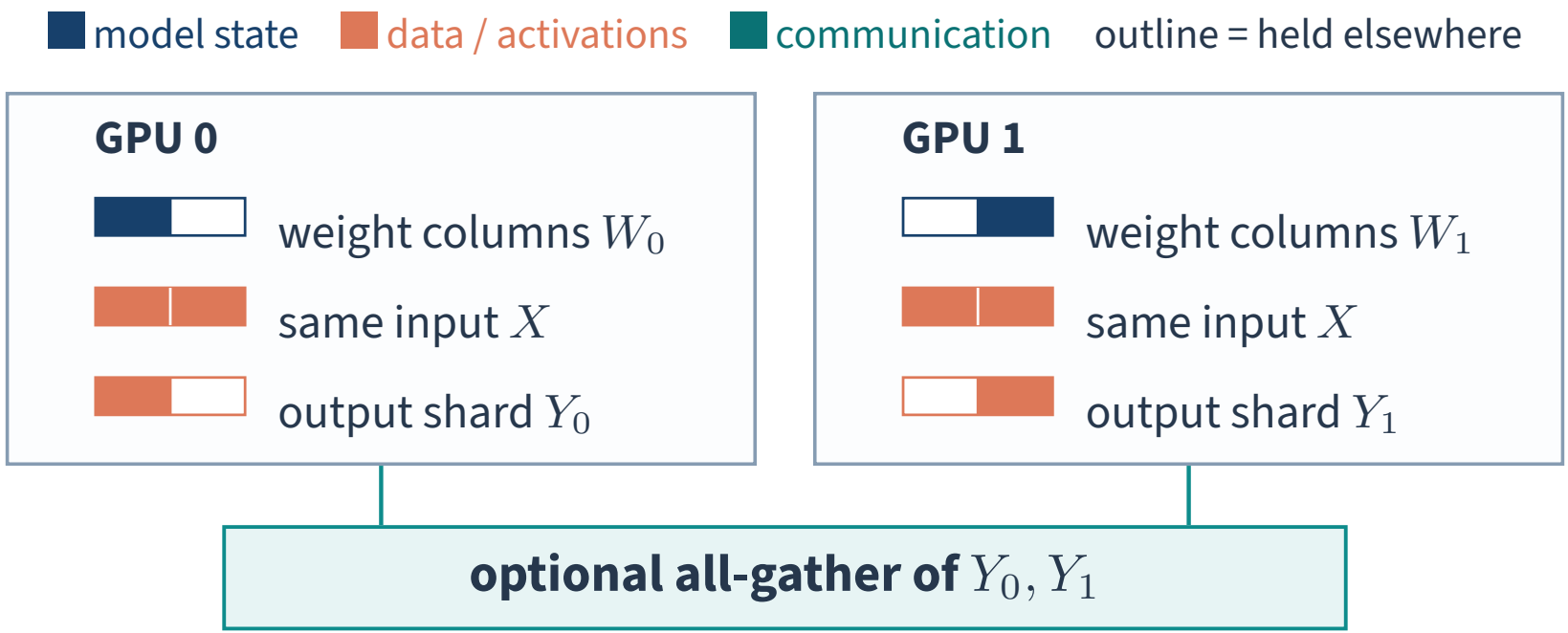
1. Derive column- and row-parallel linear layers from matrix multiplication.
2. Track replicated and sharded tensor layouts through an MLP and attention block.
3. Connect forward collectives with their backward counterparts.
4. Choose tensor-parallel degree from layer shapes and topology.

PART 1

Start from one matrix multiplication

1. Start from one matrix multiplication
2. Inside a Transformer block
3. Cost and placement

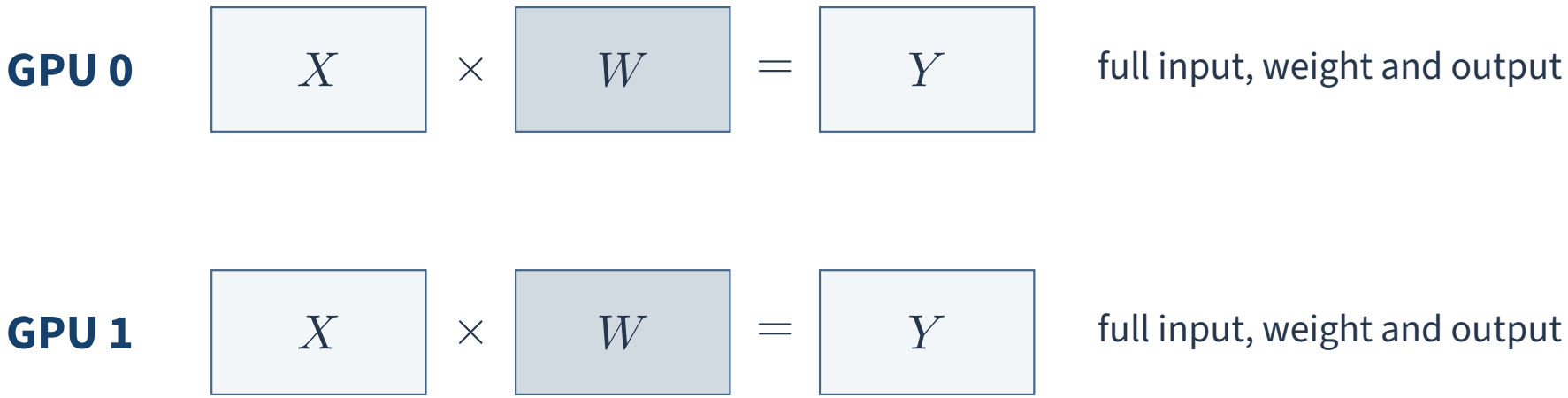
Parallelism map: TP splits a layer's width



What changes? Devices cooperate on the *same* tokens and hold different weight slices. Here $Y = [XW_0 \; XW_1]$.

A following row-parallel layer can consume sharded outputs without an intermediate gather.

First: a fully replicated matrix multiplication

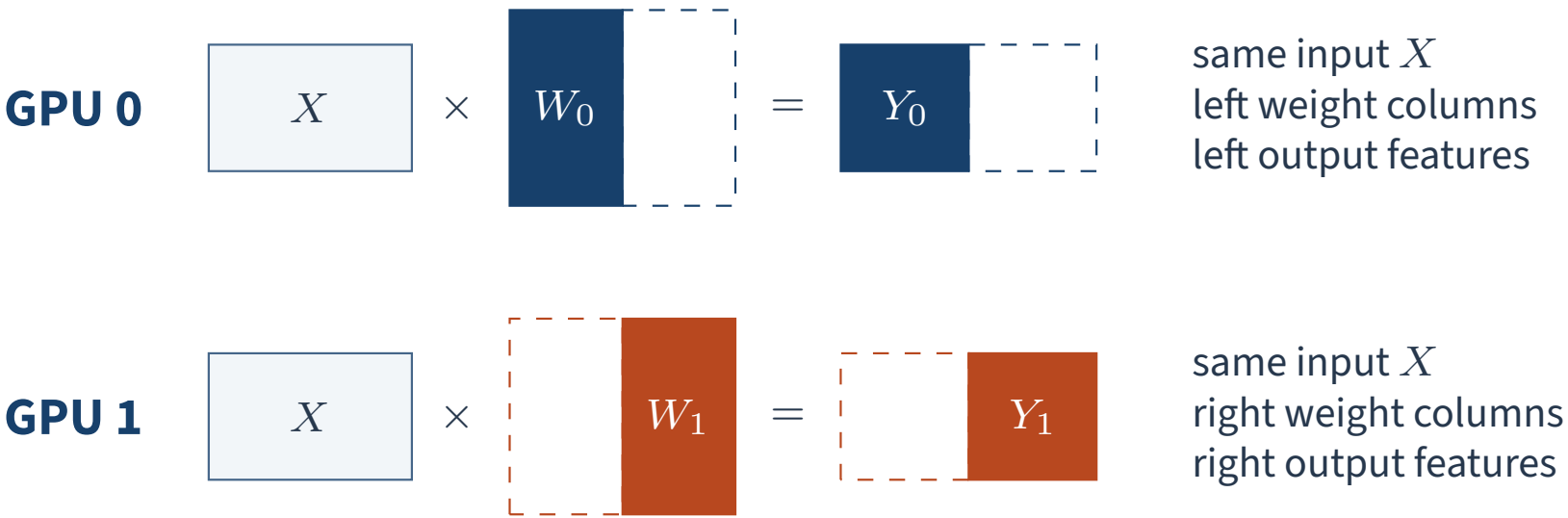


Same work on the same input

Each GPU holds the complete W and computes $Y = XW$. With identical X, the results are identical: no tensor-parallel communication is needed.

DDP normally feeds different data to its replicated models; here we use the same X to compare one operation.

Column parallelism: split output features

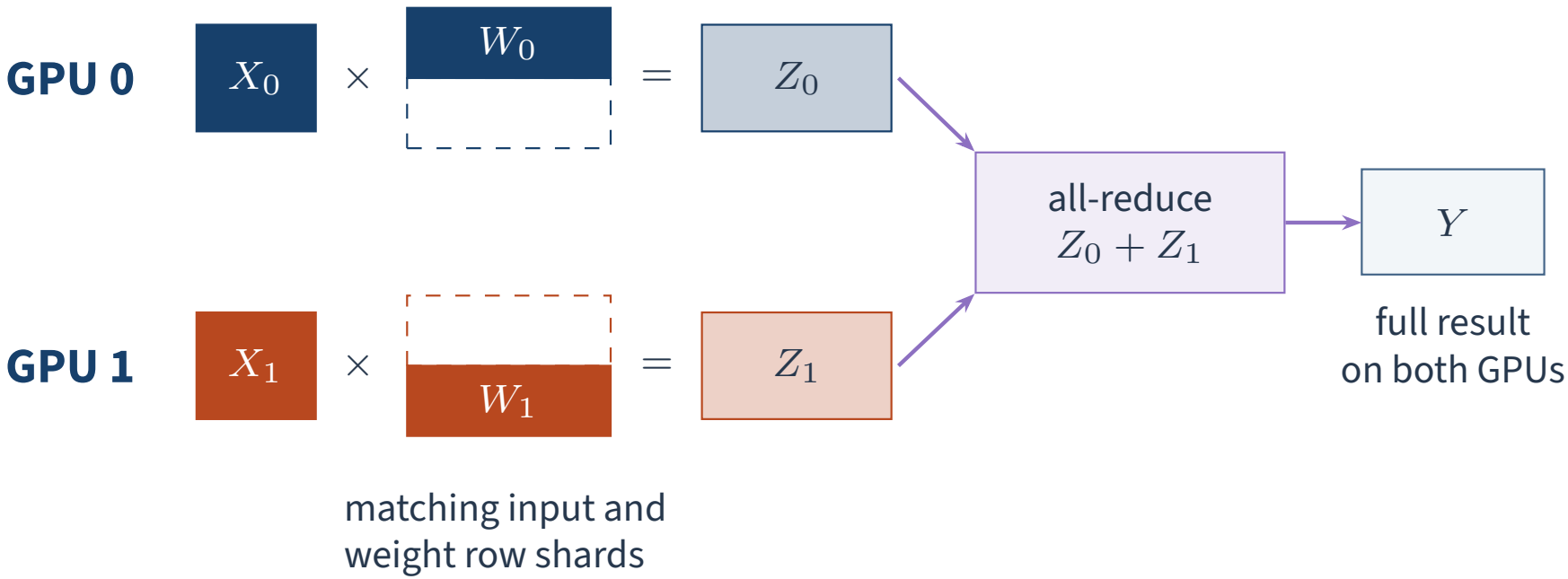


$$Y = [XW_0 \mid XW_1]$$

No forward collective

Filled = local shard; dashed = held by the other GPU. Both GPUs process the same tokens.

Row parallelism: sum partial outputs



$Y = X_0W_0 + X_1W_1$: each Z_i has the full output shape but only a *partial value*.

Three ways to distribute the same matmul

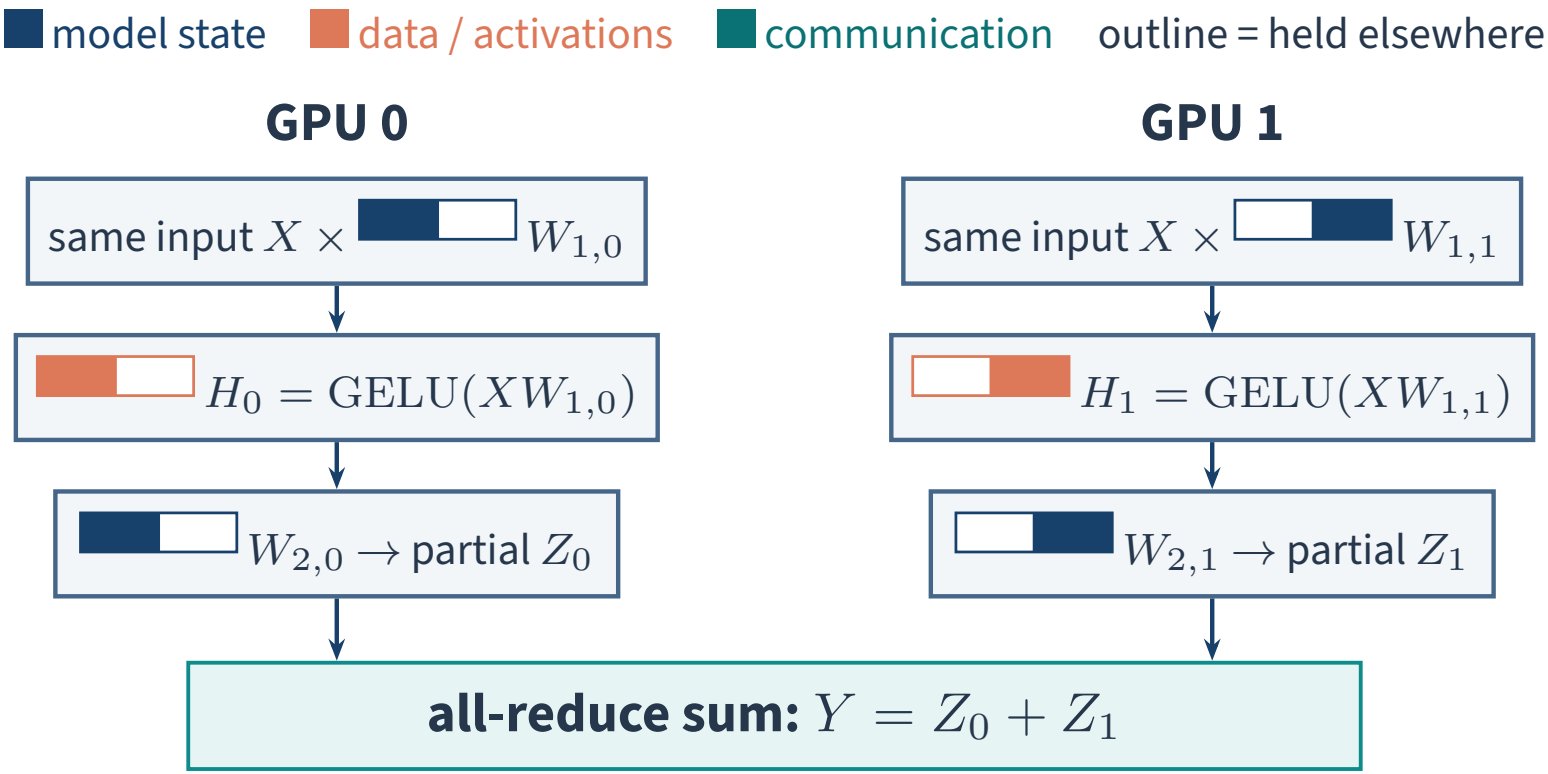
	GPU 0	GPU 1	
Replicated	$XW = Y$	$XW = Y$	two full results same work
Column	$XW_0 = Y_0$	$XW_1 = Y_1$	distinct features $Y = [Y_0 \mid Y_1]$
Row	$X_0W_0 = Z_0$	$X_1W_1 = Z_1$	partial values $Y = Z_0 + Z_1$

The distinction

Column shards hold complete values for different features. Row shards hold partial contributions to every output feature and need a sum for Y.

Forward pass, two GPUs; inputs are assumed to have the layouts shown.

Megatron MLP across two GPUs

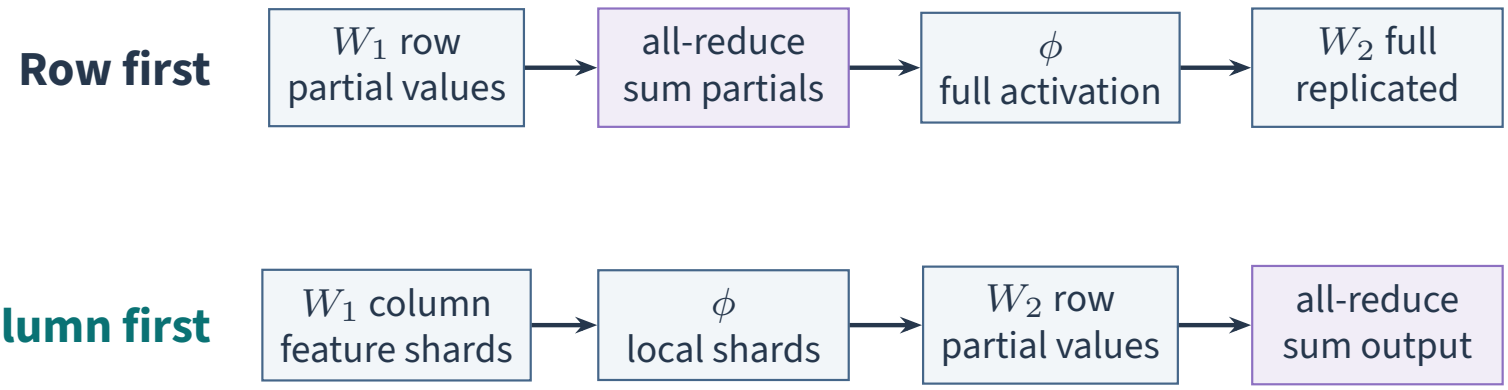


Keep H_0, H_1 sharded; sum partial outputs before the residual.

Backward sums input gradients. First bias is sharded; output bias is added once after the sum.

Source: Shoeybi et al., Megatron-LM (2019)

Why start the MLP with column parallelism?



Nonlinearity determines where the sum must happen

For GELU, in general $\phi(Z_0 + Z_1) \neq \phi(Z_0) + \phi(Z_1)$. Row first is valid only if its partial outputs are combined before ϕ.

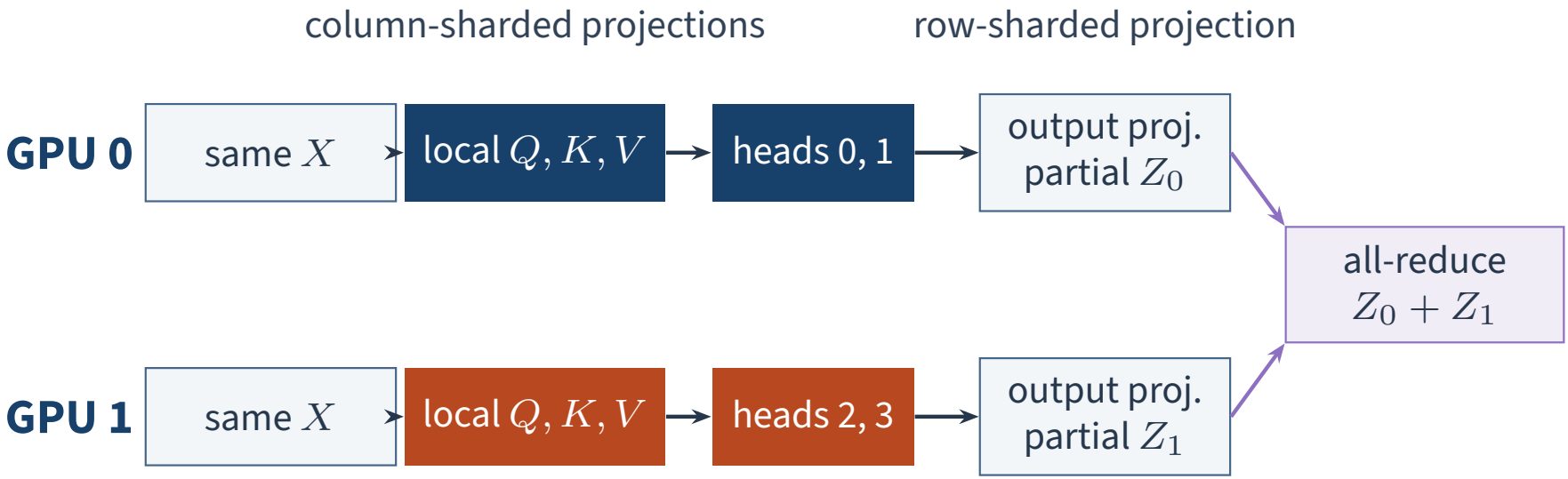
Top: assume sharded input and a replicated second layer. Bottom: sharded intermediate, one forward reduction at the output; backward also communicates.

PART 2

Inside a Transformer block

1. Start from one matrix multiplication
2. Inside a Transformer block
3. Cost and placement

Megatron attention: keep different heads on each GPU



Each GPU computes its heads locally on the same token sequence; the summed output is ready for the residual.

Backward sums input gradients. QKV biases follow head shards; add output bias once after the sum.

Four-head MHA; GQA may require a different KV-head layout.

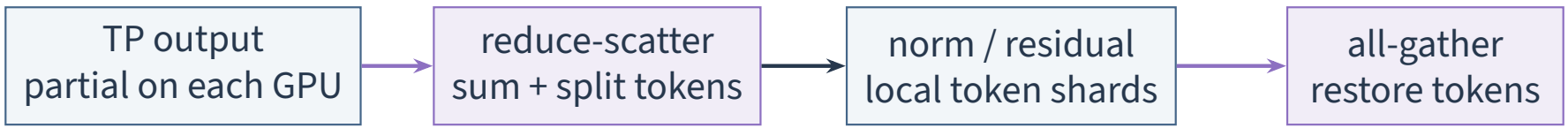
Source: Shoeybi et al., Megatron-LM (2019)

A layout ledger is the core debugging tool

Point in block	Layout	Local shape
residual input	replicated	$B \times T \times d$
QKV output	head/feature sharded	$B \times T \times 3d/p$
local attention output	head sharded	$B \times T \times d/p$
output projection partial	partial replicated shape	$B \times T \times d$
after all-reduce	replicated	$B \times T \times d$
MLP expansion	feature sharded	$B \times T \times 4d/p$

Every operation must declare what is sharded, what is replicated, and what collective changes that layout.

Sequence parallelism shards the residual stream



GPU 0: tokens 0–1 GPU 1: tokens 2–3

Memory versus communication

Shard activations between tensor-parallel regions; gather the input needed by the next region. These transitions change layout, not model math.

Example assumes per-token normalization and dropout; other operations may require communication.

Forward and backward collectives are conjugate

Forward operation	Forward layout change	Backward communication
identity with replicated input	replicated to sharded use	all-reduce input gradient
all-reduce sum	partial to replicated	identity on matching shard flow
reduce-scatter	replicated partials to shard	all-gather
all-gather	shards to replicated	reduce-scatter

Testing rule

Compare forward outputs, input gradients, and parameter gradients against an unsharded reference.

Checkpoint: layout trace

Think, then compare

Trace replicated or sharded status through a column-parallel MLP expansion, activation, row-parallel contraction, and residual add. At which point is communication unavoidable?

PART 3

Cost and placement

1. Start from one matrix multiplication
2. Inside a Transformer block
3. Cost and placement

Communication cost depends on activation size

A collective over $B \times T \times d$ elements moves bytes proportional to the activation tensor, not the parameter matrix.

$$S_{\text{act}} = B\,T\,d\,s$$

- Larger batch and context increase collective payload.
- Higher TP degree shrinks local matmuls and can reduce arithmetic efficiency.
- Collective latency and link bandwidth set the exposed cost.

Tensor parallelism belongs on the fastest links

Intra-node

High-bandwidth, low-latency GPU fabrics suit frequent activation collectives.

Across slow links

Small per-rank matmuls plus exposed all-reduces can erase the benefit quickly.

Keep TP inside a fast island when possible, then use data or pipeline parallelism across nodes.

Choose the smallest degree that solves the constraint

1. Determine whether a layer's weights or activations fail to fit on one GPU.
2. Check head, hidden, and MLP divisibility for candidate degrees.
3. Map the group to the strongest available links.
4. Measure local matmul efficiency and collective exposure.
5. Verify numerics against the unsharded block.

Checkpoint: degree selection

Think, then compare

A model fits with TP=2, but TP=8 gives lower throughput. Explain the result using local matrix shapes, collective frequency, and topology.

Key takeaways

1. Column parallelism shards output features, while row parallelism sums partial outputs.
2. Their composition keeps intermediate MLP and attention shards local.
3. A layout ledger and unsharded numerical reference are essential correctness tools.
4. Tensor parallelism is a fine-grained method that needs large local work and fast links.